

REMOÇÃO DE NÓS EM ÁRVORES AVL

DCE792 - AEDs II (Prática)

Atualizado em: 4 de outubro de 2026

Iago Carvalho

Departamento de Ciência da Computação



Uma árvore AVL é um exemplo de árvore balanceada

Ela tem seu nome devido a seus criadores

- Georgy Adelson-Velsky
- Yevgeniy Landis

Estas árvores garantem seu balanceamento utilizando a diferença entre a altura da sub-árvore a esquerda e da sub-árvore a direita

- Esta diferença nunca pode ser maior do que 1
- Diferença garantida na inserção e remoção de itens

Buscas em árvores AVL são realizadas da mesma forma que em árvores binárias

- Em ordem, pré-ordem ou pós-ordem

Na última aula, nós estudamos como inserções são realizadas

- Necessárias diversas rotações

Aqui, nós vamos estudar como é realizada a remoção de um nó

- Tema do trabalho prático

REMOÇÃO EM ÁRVORES AVL

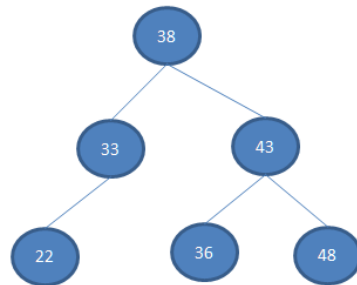
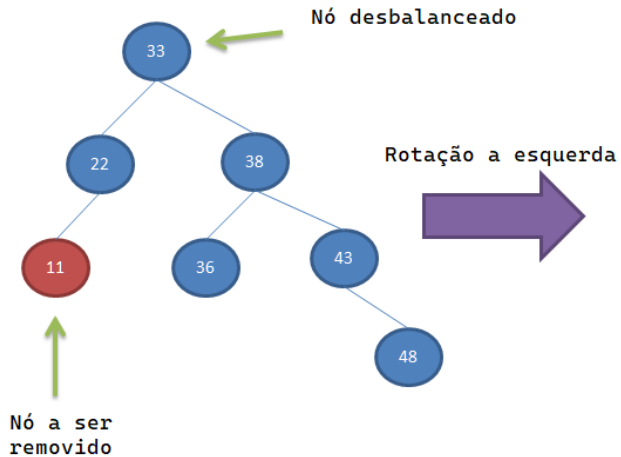
Uma remoção em árvores AVL segue alguns passos bem definidos

1. Faz-se uma remoção da mesma maneira que em uma árvore binária
2. Faz uma busca reversa (do nó removido até a raiz) recomputando o fator de balanceamento dos nós
3. Caso exista um nó não-balanceado
 - Executa as rotações necessárias
 - Quatro casos diferentes
 - Pode-se resumir a rotações a direita e a esquerda
4. Caso todos os nós estejam balanceados
 - Remoção concluída com sucesso

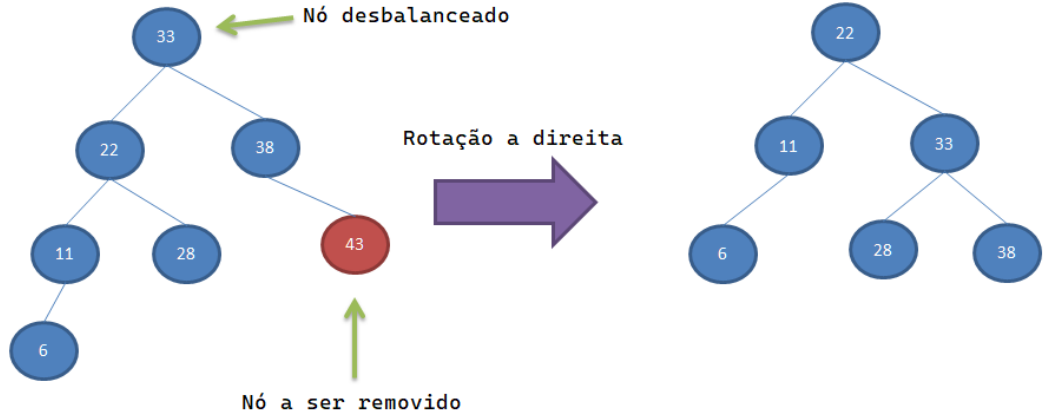
Notem que esta estrutura é muito similar ao processo de inserção

- Somente o passo 1 é diferente

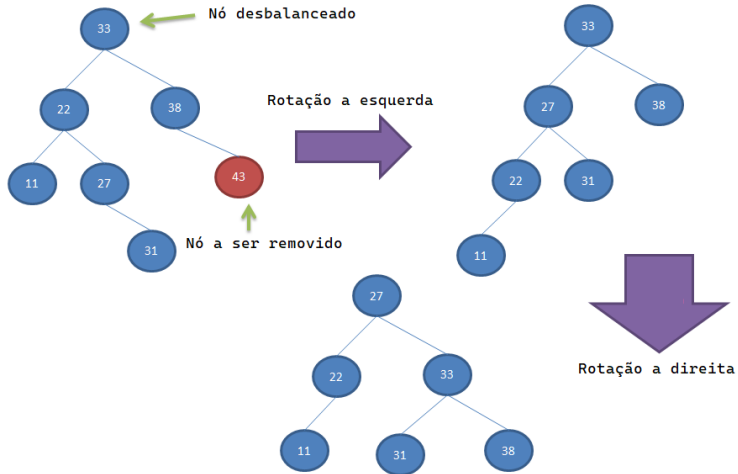
CASO ESQUERDA ESQUERDA



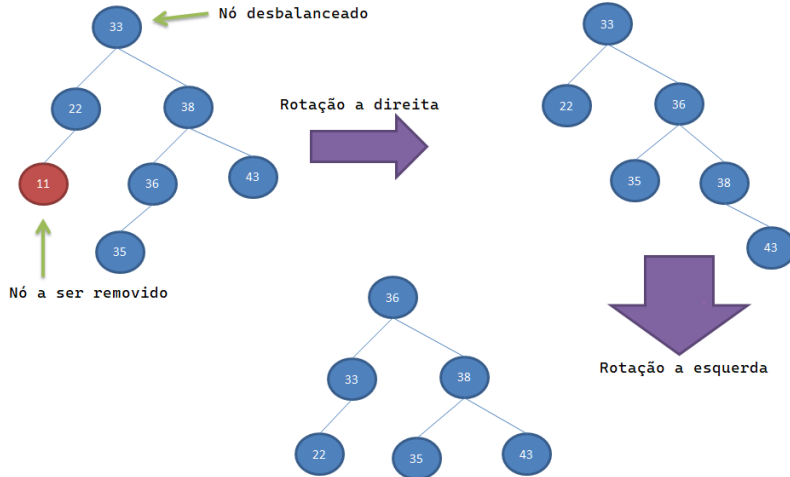
CASO DIREITA DIREITA



CASO ESQUERDA DIREITA



CASO DIREITA ESQUERDA



O código (quase) completo de uma árvore AVL está disponível no Github

- É o mesmo código que vocês utilizaram como base na aula passada
- Sem rotação a esquerda
- Sem remoção de nós

O algoritmo pode ser visualizado em [▶ Link](#)

ATIVIDADE PRÁTICA

Deve-se implementar a função de remoção de vértices

- Para isto é necessário ter implementado a rotação a esquerda

```
struct node* remover(<valor>);
```

PRÓXIMA AULA:

ALGORITMOS DE
ORDENAÇÃO